Forza Fantasy League — Technical Documentation

Complete technical reference for the multi-sport fantasy gaming platform.

This document is the detailed companion to the "Technical Overview". It is written for an engineering audience evaluating, operating, or extending the platform.

Table of contents

---

1. Introduction & product scope

Forza Fantasy League is a fantasy-sports platform delivering private-league competition across multiple sports. Users build squads, compete on live real-world match scoring, and interact through social and engagement features. The product launched as a football game and has been extended into a multi-sport platform.

Sports & game modes

- **Football** — squad of 15 (11 starters + bench), formation rules, captain, transfers, live scoring. League formats: Classic, Draft, Head-to-Head, Cup (knockout).
- **Formula 1** — "paddocks" (leagues), race-by-race scoring.
- **Tennis** — "player boxes" (leagues), tournament rosters with tier-based scoring, and an ATP Finals pick'em mini-game.

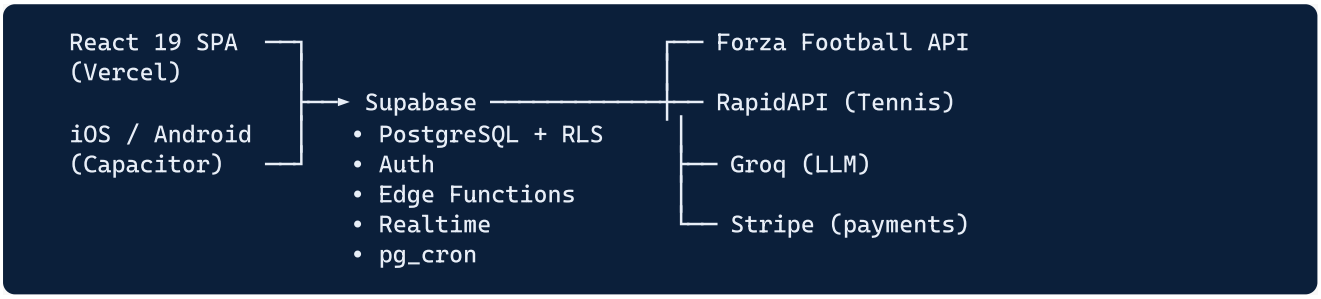
Engagement layer (cross-sport) — league chat, an AI-generated daily newspaper, trading/auctions, commissioner-run prediction bets, trophy cabinet, and a virtual coin wallet with Stripe top-ups.

---

2. System architecture

2.1 High-level shape

The platform is a **client → Supabase** architecture. There is no separate bespoke application server; backend logic lives in (a) PostgreSQL functions invoked as RPCs and (b) Supabase Edge Functions (Deno). The React client talks to Supabase over HTTPS and WebSocket (Realtime).



2.2 Design principles

- **Server-authoritative money & points.** Anything affecting budget, points, coins, or league outcomes is computed server-side (RPC or Edge Function), never trusted from the client.
- **Row-Level Security everywhere on user data.** Each query is scoped to `auth.uid()`.
- **Idempotent pipelines.** Scoring and sync jobs can be safely re-run; results converge.
- **Append-only auditability.** Squad mutations, round snapshots, and ledger entries are recorded for recovery and dispute resolution.

---

3. Frontend application

Aspect	Detail
Framework	React 19 (functional components + hooks)
Build tool	Vite 8 (Rolldown — Rust-based bundler)
Styling	Tailwind CSS 4 + a CSS-variable design-token system ("Kit Light") + inline styles for dynamic values
Routing	React Router 7 (hash-compatible for Capacitor)
State	React Context (`AuthContext` , `SportContext`) + per-feature hooks; data fetched directly via the Supabase client
Real-time	Supabase Realtime subscriptions (chat, live scores)

Structure

- `src/screens/` — route-level views (football, plus `f1/` and `tennis/` subtrees, admin, social).

- `src/components/` — reusable UI; `components/league/` holds the larger league sub-views (commissioner panel, recap, stats, trading, chat).
- `src/hooks/` — feature logic (auth, squad, transfers, league config, scoring stats, plus `f1/` and `tennis/` hook subtrees).
- `src/lib/` — Supabase client, Capacitor init, API helpers, formatters.
- `src/context/` — global providers.

Resilience — each route is wrapped in an `ErrorBoundary` with a client-side crash reporter, limiting the blast radius of any runtime error to a single screen.

---

4. Backend (Supabase)

Supabase is the managed backend. It provides:

Capability	Usage in this platform
PostgreSQL	All persistent data; business logic in SQL functions (RPCs)
Auth	Email/password (OAuth-ready); sessions issued as JWTs
Edge Functions	Deno/TypeScript serverless functions for scoring, syncs, payments, AI generation
Realtime	LISTEN/NOTIFY → WebSocket for chat & live updates
pg_cron	Scheduled jobs: fixture/player sync, live ingest, scoring passes, bet/auction resolution
Row-Level Security	Per-user data isolation enforced at the database layer

Business logic in the database. Critical operations are implemented as `SECURITY DEFINER` PostgreSQL functions so they run with owner privileges and bypass RLS in a controlled, audited way. Examples: `execute_transfer_atomic`, `set_lineup`, `set_captain`, `place_bid`, `confirm_auction_win`, `accept_trade_proposal`, `resolve_bet`, `claim_draft_player`, `credit_coins` / `debit_coins_to_escrow` / `release_escrow`.

---

5. Data model & key entities

A non-exhaustive map of the principal tables (229 migrations define the full schema):

Identity & leagues - `users` — profile + flags (incl. `is_admin`). - `leagues`, `league_members`, `league_config` — league definitions, membership/roles, per-league settings (caps, transfer rules, scoring weights, H2H points).

Squads & play - `squads` — roster (`players`), `starting_xi`, `captain_id`, `budget_remaining`, `round_transfers`, `lineup_locks`, `initial_build_complete`. - `draft_submissions`, `draft_allocations`, `knockout_keep_submissions` — draft inputs/outputs. - `transfer_windows` — manual/automatic transfer-window control.

Fixtures & scoring - `fixtures`, `match_events`, `player_match_stats` — match data and per-player statistics. - `scoring_rules`, `matchday_deadlines`, `club_cap_rules` — scoring weights and round

configuration. - `fantasy_points` — points per `(squad_id, matchday_id)`.

Competition & social - `h2h_schedule`, `gazette_entries` (activity & news feed), `bet_instances` / `bet_submissions`, `auction_listings`, `trade_proposals`. - `frontpage_editions` / `frontpage_reactions` / `frontpage_comments` — AI newspaper.

Money (P2P) - `coin_wallets`, `coin_transactions`, `coin_packs` — wallet, audited ledger, purchasable packs.

Multi-sport - F1: `f1_paddocks`, `f1_races`, `f1_scores`, `f1_year_results`. - Tennis: `player_boxes`, `tennis_tournaments`, `tennis_rosters`, `tennis_tournament_players`, `tennis_ace_cards`, `tennis_atp_finals_*`.

Audit & recovery - `squad_events` (append-only mutation log), `round_backups` (per-round full snapshot), `squad_matchday_snapshots` (XI at round start), `edge_function_error_log`.

---

6. Core domain systems

6.1 Scoring engine (`calculate-scores`)

Converts `player_match_stats` into fantasy points using position-specific `scoring_rules`. Applies per-90/per-60 minute scaling, clean-sheet gates, captain multipliers, and **auto-substitutions** (a non-playing starter is replaced by the highest-priority bench player who played, keeping the formation valid). Captain bonus reassigns to a scoring starter if the captain isn't in the effective XI. Writes integer totals to `fantasy_points`, keyed `(squad_id, matchday_id)`. **Idempotent** per fixture and guarded against re-scoring a settled round.

6.2 Squad & transfer engine (`execute_transfer_atomic`)

Atomic, server-side, advisory-locked. Enforces budget, squad size (15), position caps, club caps (round-aware), and the transfer window. Buys count toward the per-round limit; sells are free; over-limit buys are allowed but recorded as point penalties. A one-way `initial_build_complete` latch exempts the initial squad build and pre-competition period from limits.

6.3 Lineup & captain (`set_lineup` , `set_captain`)

Atomic swaps of starters/bench with per-player fixture-status locking (you can't pull a player out after their match starts) and live-mid-round point recomputation. Captaincy cannot be reassigned to a player whose fixture has started.

6.4 Draft system

Lottery (`run-draft-lottery`) and reverse-standings (`run-reverse-standings-draft`) modes. Supports wishlists, club-cap enforcement during allocation, knockout-phase "keep" pre-allocation, and recovery for late joiners. Draft picks go through `claim_draft_player` (advisory-locked, no double-claims).

6.5 Trading & auctions

`submit_trade_proposal` / `accept_trade_proposal` (same-position validation, window check, points sweetener transfer, gazette entry). Auctions are two-phase: a listing resolves to `pending_confirmation` on deadline, and the winner confirms during an open window (`confirm_auction_win`), with budget/slot/duplicate checks at confirmation time.

6.6 Betting (`resolve_bet` , `resolve-bets`)

Commissioner-created prediction bets (match result, clean sheet, etc.). Commissioners can resolve at any time; an auto-resolve cron handles deadline-passed bets. Winning "points" bets immediately re-aggregate the affected managers' league totals.

6.7 Coin wallet (P2P)

`coin_wallets` + audited `coin_transactions` ledger with escrow. Top-ups via Stripe (`purchase-coins` Edge Function): server-side price lookup, JWT-derived user, webhook-verified fulfilment, idempotent on `reference_id` . Mutating coin RPCs are revoked from client roles.

6.8 AI newspaper (`generate-frontpage-edition`)

A daily per-league "front page" generated by a Groq LLM from standings, recent transfers, chat, fixtures, and gazette entries. Members react (emoji) and comment ("letters to the editor"). Commissioners can post special editions and pin a quote.

6.9 Multi-sport modules

- **F1** — paddocks (leagues), race scoring (`score-f1-race`), season results.
- **Tennis** — player boxes, tier-based tournament scoring (`score-tennis-tournament`), ace-card power-ups, a Masters drop rule on the leaderboard, and an ATP Finals 15-match pick'em (`score-atp-finals`).

---

7. The live data & scoring pipeline

Stage	Job(s)	Cadence
Sync fixtures/players	`sync-fixtures`, `sync-players`, `sync-player-status` (football); `sync-tennis-players` (admin-triggered)	every 30 min (football)
Flip to live	`flip-fixtures-live`	every 2 min
Ingest events	`ingest-match-events`	every ~5 min while live (+ a final post-whistle pass)
Score (live)	`calculate-scores` (live mode)	every 2 min for live fixtures
Score (post-match)	`calculate-scores` (post-match)	daily, 24h window
Score (late finishers)	`calculate-scores` (late)	nightly, 3h window
Settle	round-complete branch of `calculate-scores`	when all round fixtures finish
Resolve bets / auctions	`resolve-bets`, auction sweep	hourly / every 5 min

Round settlement writes the activity feed entry, resolves head-to-head results, applies auto-subs and transfer penalties, and stores a `round_backups` snapshot — all gated on every fixture in the round being finished, so multi-day rounds are handled correctly.

Resilience — idempotent writes (upserts) plus overlapping windowed schedules mean a missed or partial run is recovered by a later pass.

---

8. Edge Functions reference

21 Deno/TypeScript functions in `supabase/functions/`:

Function	Role
`calculate-scores`	Core fantasy-points engine (football)
`calculate-relaxation`	No-repeat relaxation as the cup player pool shrinks
`ingest-match-events`	Live event ingestion + fixture-status flips
`sync-fixtures` / `sync-players` / `sync-player-status`	Football data sync from Forza
`discover-tournament`	Tournament onboarding from the API
`process-transfer`	Transfer orchestration wrapper around the atomic RPC
`run-draft-lottery` / `run-reverse-standings-draft`	Draft allocation engines
`resolve-bets`	Auto-resolution of deadline-passed bets
`eliminate-cup-club`	Cup elimination handling
`auto-open-transfer-window`	Scheduled transfer-window opening
`generate-frontpage-edition`	AI newspaper generation (Groq)
`purchase-coins`	Stripe coin purchase + webhook fulfilment
`score-f1-race` / `score-tennis-tournament` / `score-atp-finals`	Multi-sport scoring
`sync-tennis-players`	Tennis roster sync (RapidAPI, admin-triggered)
`test-forza-api`	Diagnostic
`_shared`	Shared helpers, incl. `auth.ts` (`requireServiceRole` , HMAC verification)

Deployment note: Edge Functions are **not** auto-deployed with the frontend — they are deployed manually (`npx supabase functions deploy <name>`). This is operationally significant (see §17).

---

9. Security model

Control	Implementation
Authentication	Supabase Auth (JWT sessions); frontend gated by `VITE_AUTH_ENABLED`
Authorization (data)	Row-Level Security on user tables, scoped to `auth.uid()`
Authorization (logic)	`SECURITY DEFINER` RPCs run as owner; mutating money/points RPCs revoked from `anon` / `authenticated`
Protected columns	Triggers (`guard_squad_protected_columns` , coin-wallet guard) block clients from writing budget/identity/ledger fields directly — those change only via RPC
Service-role verification	`_shared/auth.ts` provides HMAC-SHA256 verification of service-role tokens
Payments	Stripe webhook signature verification; server-side price/coin lookup; user identity from JWT
XSS	No `dangerouslySetInnerHTML` / `eval` ; user content rendered as escaped JSX

> The security posture is strong in its design (RLS, protected-column triggers, server-authoritative money). A small number of server endpoints need the existing `requireServiceRole` helper applied and one client-writable flag locked down — these are itemised in the remediation backlog and are quick fixes.

10. External dependencies

Service	Purpose	Notes
Forza Football API	Football fixtures, players, live events	Core, sole football data feed
RapidAPI (ATP/WTA/ITF Tennis)	Tennis player/tournament data	Free tier (50 req/day) — admin-triggered, ~28 calls/season; move to paid plan before scale
Groq	LLM for the AI newspaper	`llama-3.1` , OpenAI-compatible API
Stripe	Virtual coin purchases	Webhook-verified fulfilment
Supabase	Database, Auth, Functions, Realtime, cron	Managed backend platform
Vercel	Frontend hosting + auto-deploy	From the football `main` branch

All third-party secrets are stored as Supabase Edge Function secrets / Vercel env vars (not in source).

11. Mobile (Capacitor)

A single web codebase is wrapped natively via **Capacitor 8** for iOS and Android.

- App ID: `com.fantasykit.forzaedition`.
- Native plugins: status bar, splash screen, app-resume handling (`src/lib/capacitor.js`).
- Build: `npm run build && npx cap sync` copies the web build into the native projects.
- **Status:** the native projects compile in CI (iOS simulator / Android debug, unsigned). They are **not yet store-submitted** — signing certificates, provisioning, release builds, and store listings are roadmap items. Mobile should currently be regarded as a near-ready wrapper, not a shipped app.

---

12. Build, CI/CD & deployment

Stage	Detail
Frontend build	`vite build` → `dist/`
CI (GitHub Actions)	On PR: ESLint, production build, Playwright UI smoke suite (desktop + mobile viewports)
Frontend deploy	Vercel auto-deploys on merge to `main` (~30s)
Edge Function deploy	**Manual** — `npx supabase functions deploy <name>`
DB migrations	Versioned SQL files, applied via Supabase CLI
Branching	`main` (live football pilot) + `v2` (multi-sport platform); feature branches → PR → squash-merge

---

13. Environments & configuration

Single environment today — the live Supabase project is also the development/pilot database. Standing up a staging environment is a recommended early step for a new owner (see backlog).

Frontend env vars (Vercel): - `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` — client connection. - `VITE_AUTH_ENABLED` — must be `"true"` in production (enables real auth).

Backend secrets (Supabase): `FORZA_ACCESS_TOKEN`, `RAPIDAPI_TENNIS_KEY`, `GROQ_API_KEY`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `SUPABASE_SERVICE_ROLE_KEY`.

Ownership transfer requires: a new Supabase project (+ re-keyed secrets), a new Vercel project, a new GitHub repo, and the buyer's own Forza/Stripe/Groq/RapidAPI accounts. The current Supabase project reference is embedded in several source files and migration cron URLs — externalising it is a documented backlog item.

---

14. Testing

Layer	Coverage
E2E (CI)	`platform.spec.js` — UI render/route smoke suite (Playwright), desktop + mobile
E2E (manual)	Integration specs for draft/scoring/bets — currently run manually against live data
Unit / RPC / function	None currently automated

Expanding automated coverage of the money/scoring logic (against a seeded staging DB) is the highest-value engineering investment and is the lead item in the hardening roadmap.

---

15. Observability & operations

- **Logging:** Edge Functions log via console (visible in Supabase logs).
- **Error capture:** an `edge_function_error_log` table + a `cron_job_status()` RPC + an admin error-monitor panel exist.
- **Alerting:** not yet implemented (no Sentry/external alerting) — a roadmap item.
- **Backups:** currently manual; enabling Supabase PITR + automated off-site backups is recommended early.
- **Runbooks:** the repo includes deployment, data-pipeline, key-rotation, and new-tournament runbooks under `docs/deployment/`.

---

16. Repository structure

```
src/                  React app (screens, components, hooks, lib, context)
  screens/            route views (+ f1/, tennis/ subtrees)
  components/league/  large league sub-views
  hooks/              feature logic (+ f1/, tennis/)
  lib/                supabase client, capacitor, helpers
supabase/
  migrations/         229 versioned SQL files
  functions/          21 Deno Edge Functions (+ _shared)
e2e/                  Playwright specs
ios/ android/         Capacitor native projects
docs/                 architecture, api, brand, deployment, handover
.github/workflows/    CI (web build/lint/test, mobile build)
```

---

17. Known limitations & hardening roadmap

The platform is functional and feature-complete across three sports. The following are the known areas to harden as it moves from pilot to scale-ready commercial product. **Full detail, effort estimates, and exact file locations are in the companion "Remediation Backlog".** Summary by theme:

- **Security (quick fixes):** apply the existing service-role verification to a few scoring endpoints; lock down one client-writable admin flag; harden the (not-yet-live) payment path; rotate a repo access

token.

- **Environment & data:** create a reproducible schema baseline; provision staging; enable point-in-time recovery and automated backups.
- **Testing:** automated coverage of the core money/game-logic RPCs against a seeded DB.
- **Delivery:** gate Edge Function + migration deploys so frontend/backend can't drift; add dependency/secret/security scanning to CI.
- **Observability:** add error tracking and cron/failure alerting.
- **Maintainability:** introduce a data-fetching layer; break up the largest screen components; extract a shared multi-sport abstraction; adopt incremental typing.

These are typical of a successful pilot transitioning to scale and are individually tractable; none require re-architecting the platform.

---

18. Glossary

Term	Meaning
Matchday / round	A scoring period (`{tournament}-r{N}`); points are keyed per squad per round
Gazette	The in-app activity & news feed
Paddock / Player Box	The F1 / Tennis equivalents of a "league"
Chip	A power-up (e.g. triple captain) — currently disabled for the football pilot via a feature flag
RPC	A PostgreSQL function called from the client (server-authoritative logic)
RLS	Row-Level Security — per-row access control in PostgreSQL
Edge Function	A Deno serverless function hosted by Supabase
Idempotent	Safe to run multiple times with the same result

Companion documents: "Technical Overview" (high-level) · "Remediation Backlog" (engineering roadmap). Last updated: 2026-06-26.
